Programmation 3D
Notions de base

Objectif

Ce cours a pour objectif de présenter les différentes notions de bases entrant en jeu dans un moteur 3D.

Surface de rendu

Surface de rendu

Une surface de rendu, ou **frame buffer**, est une matrice de données rectangulaire sur laquelle sera calculée le rendu à afficher.

Pour chaque pixel, le frame buffer contient :

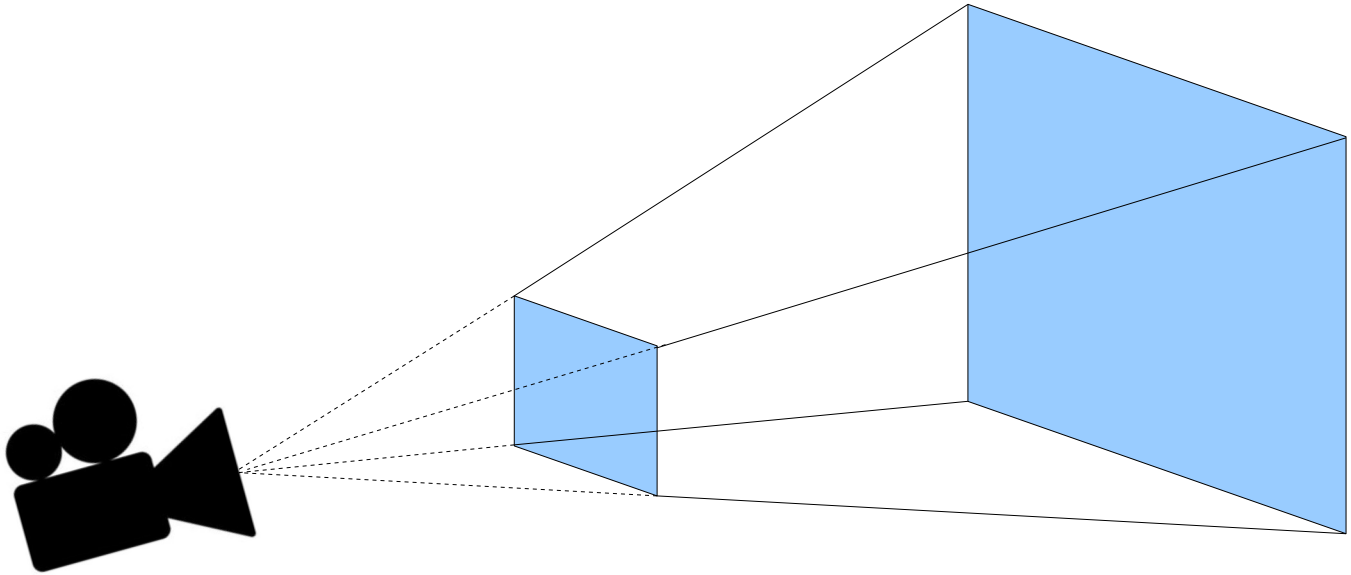
- La couleur du pixel,
- La profondeur du pixel dans la scène,
- Des données additionnelles du pixel.

Les dimensions du frame buffer sont généralement celle de la résolution écran.

Caméra

Caméra

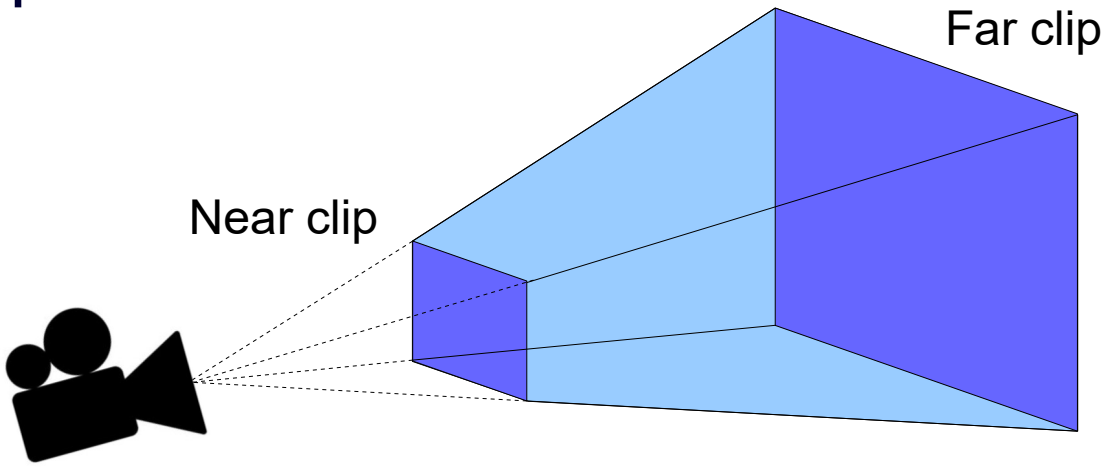
La **caméra** est le point de vue sur la scène à afficher.



Caméra : Frustum

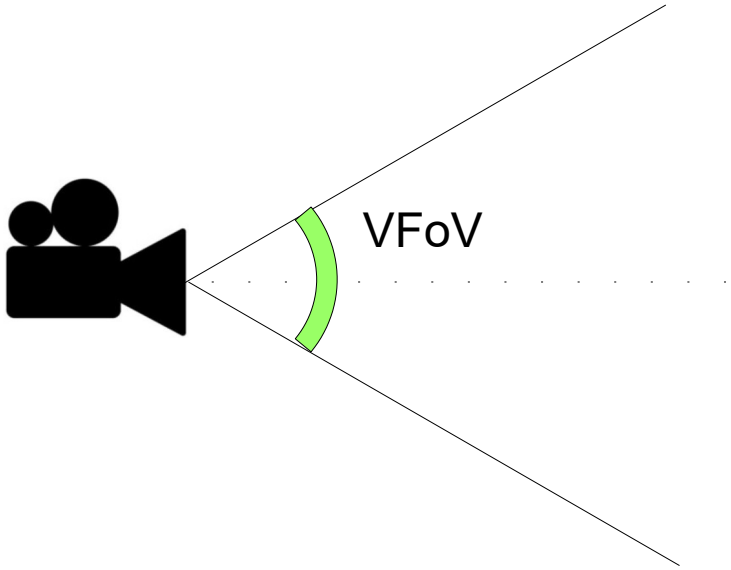
L'espace rendu par la caméra est compris dans un "**frustum**", une pyramide tronquée par deux plans :

- Le "near clip"
- Le "far clip"



Caméra : Champ de vision

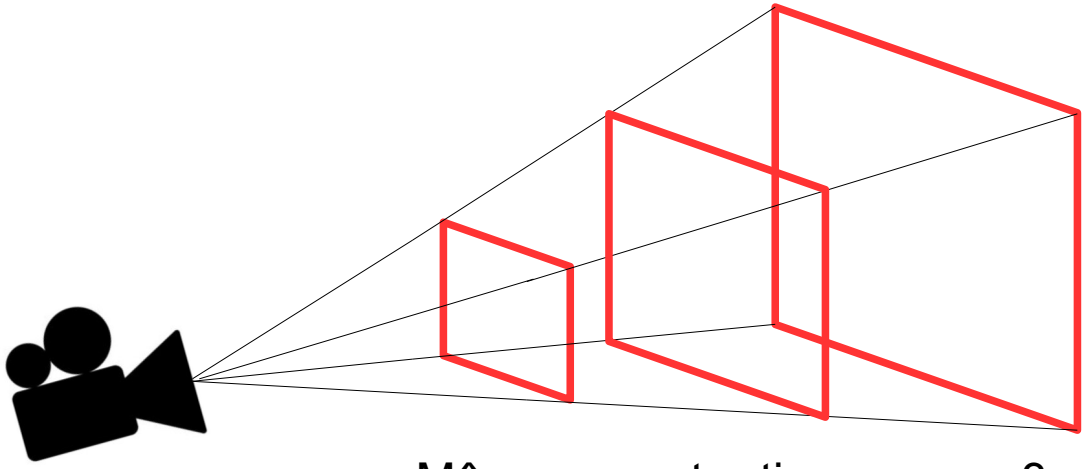
Le champ de vision vertical, ou **VFoV**, est l'angle vertical au sommet de la pyramide.



Caméra : Aspect Ratio

Le rapport hauteur-largeur, ou **aspect ratio**, dépend la taille de la surface de rendu. Sur n'importe quel plan perpendiculaire au frustum, il vaut :

Aspect Ratio = Width / Height

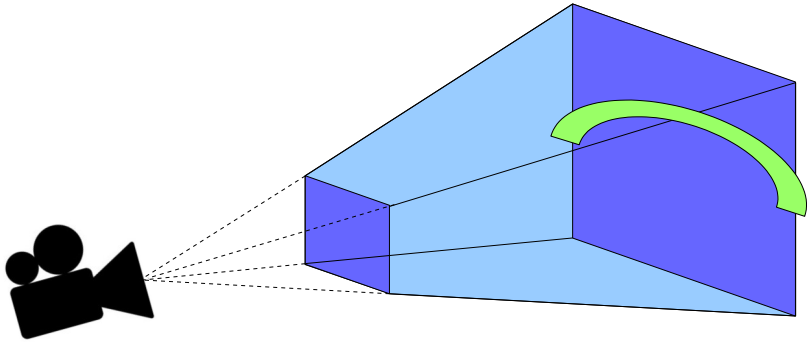


Même aspect ratio pour ces 3 coupes !

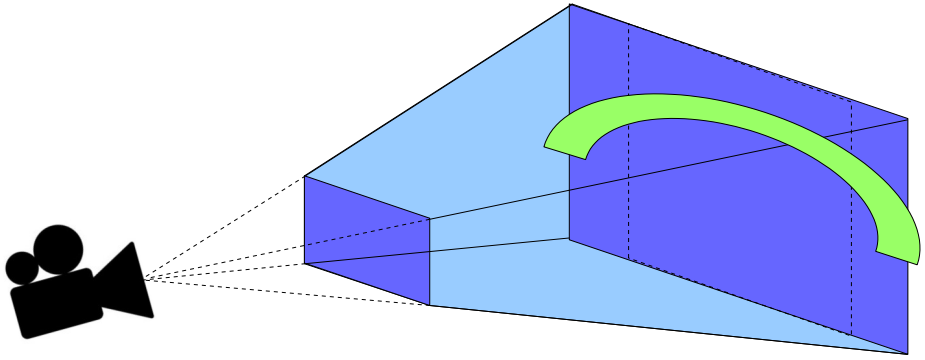
Caméra : Aspect Ratio

Le VFoV et l'Aspect Ratio permettent de calculer le champ de vision horizontal, ou HFoV :

$$aspectRatio = \tan(HFoV / 2) / \tan(VFoV / 2)$$



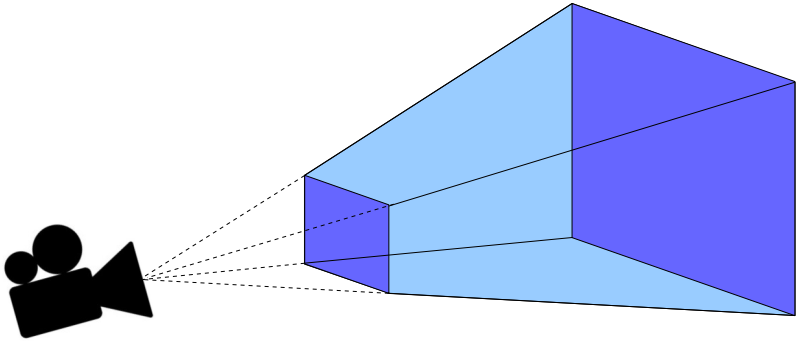
Aspect ratio : 4/3



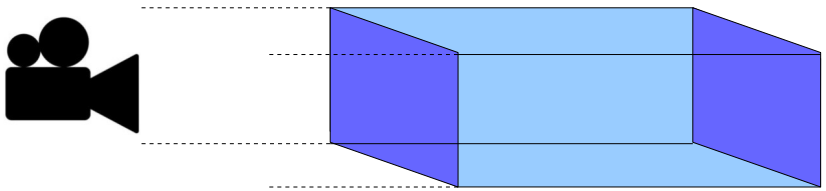
Aspect ratio : 16/9

Caméra : Projections

La projection de la caméra définit comment la perspective de la scène 3D est rendue :



Projection en perspective

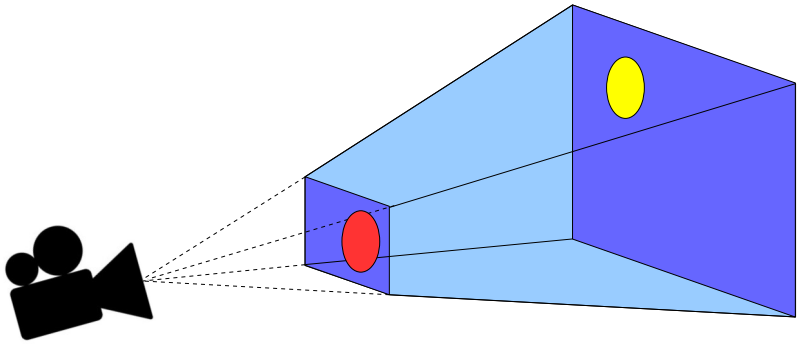


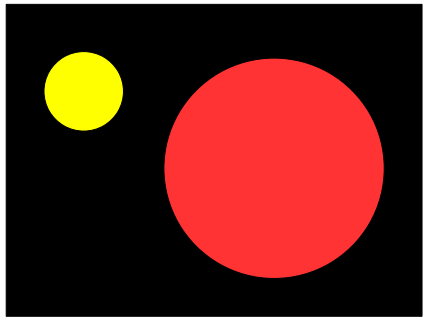
Projection orthogonale

Caméra : Projections

Projection en perspective :

- Plus une coupe du frustum est loin, plus elle est grand.
- Plus un objet est loin, plus il est affiché petit.

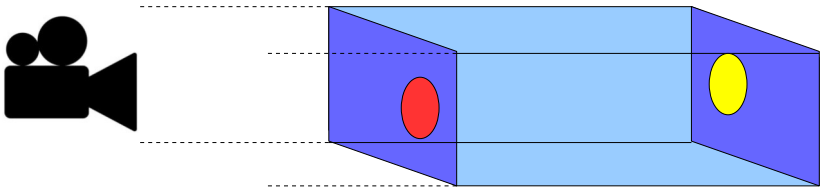


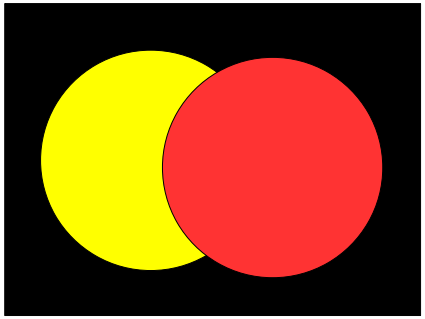


Caméra : Projections

Projection orthogonale :

- Toutes les coupes du frustum ont la même taille.
- La taille d'affichage d'un objet ne dépend pas de sa distance à la caméra.





Éclairage

Lumière

Une source lumineuse se définie par :

- Une couleur
- Une intensité
- Un champ de vecteur

Lumière

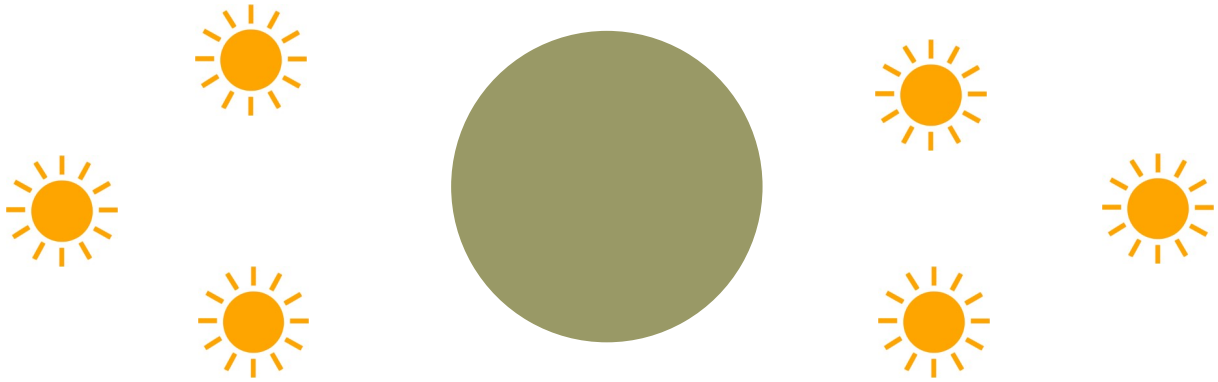
Les principaux types de sources lumineuses en 3D sont :

- La lumière ambiante
- La lumière directionnelle
- Les "point lights"
- Les "spot lights"

Lumière ambiante

La lumière ambiante est sans orientation et unie sur toute la scène.

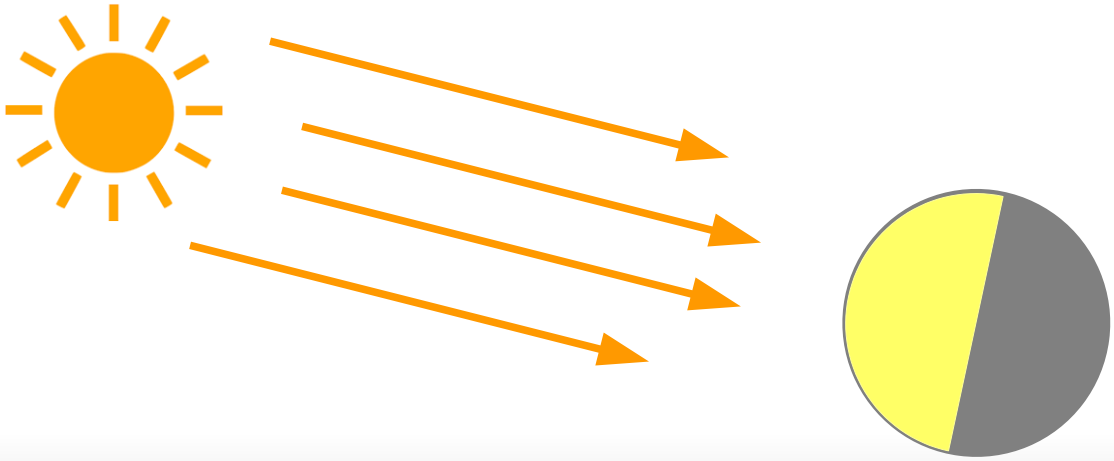
Elle représente une lumière faible propagée suite à de multiples rebonds.



Lumière directionnelle

La lumière directionnelle est unidirectionnelle et présente sur toute la scène.

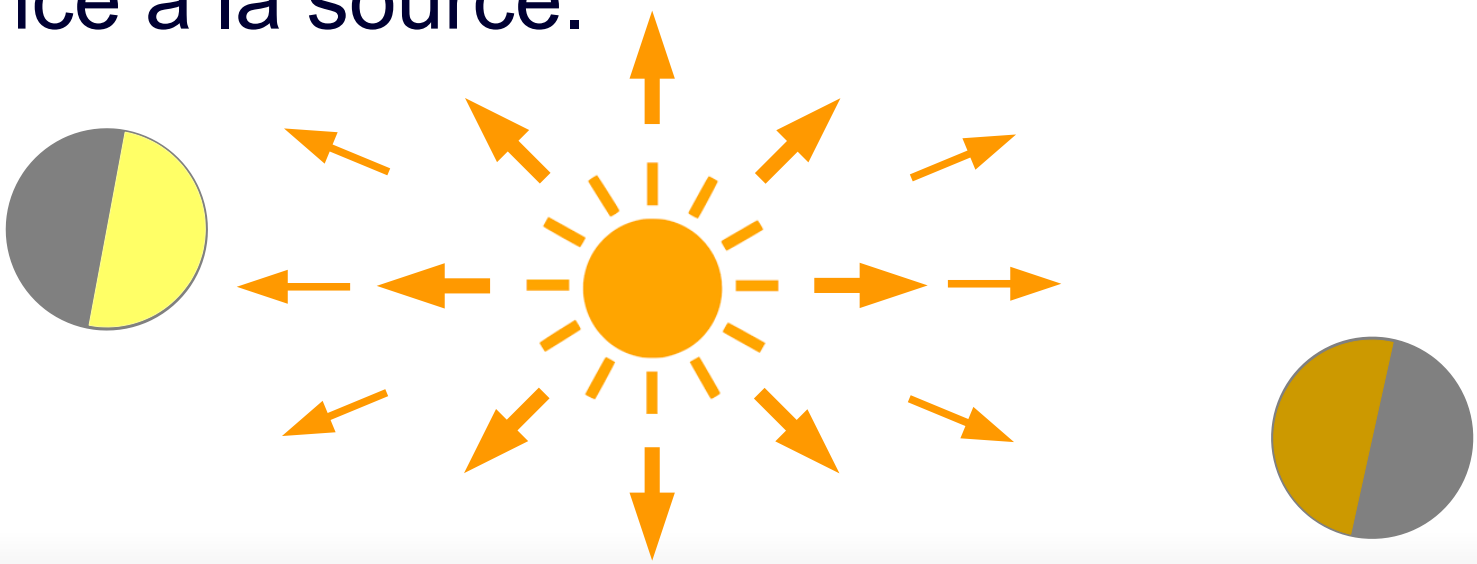
Elle représente une source de lumière dite "à l'infini" telle que le soleil.



Point-Light

Un "point-light" est émis à partir d'un point de l'espace et rayonne dans toutes les directions.

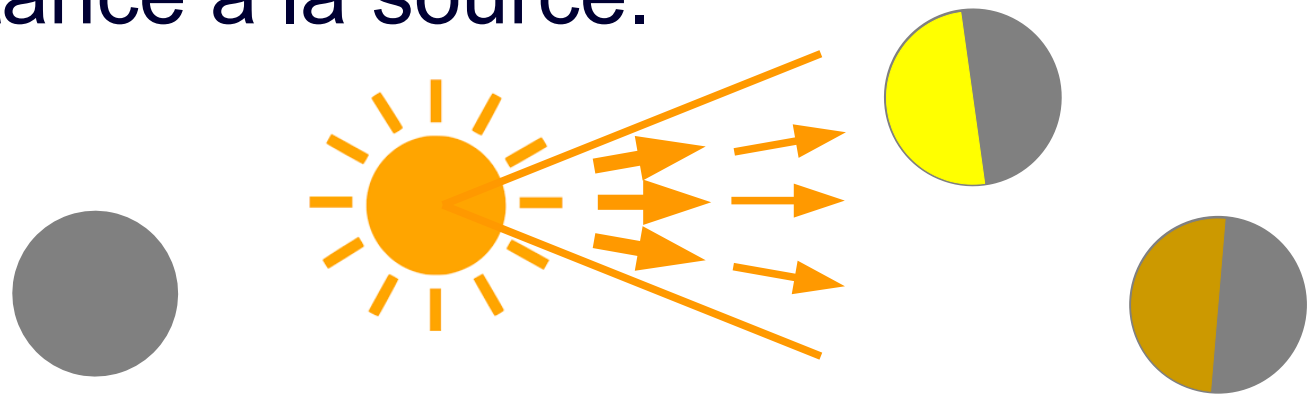
Son intensité s'atténue en fonction de la distance à la source.



Spot-light

Un "spot-light" est émis à partir d'un point de l'espace et se diffuse en cône dans une direction donnée.

Son intensité s'atténue en fonction de la distance à la source.



Objets 3D

Objet 3D

Un objet 3D est composé de :

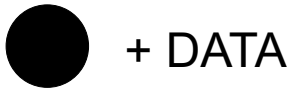
- Meshes
  - Vertices
- Materials
  - Shader
  - Textures
  - Paramétrage
- Données d'animation

Meshes

Vertex

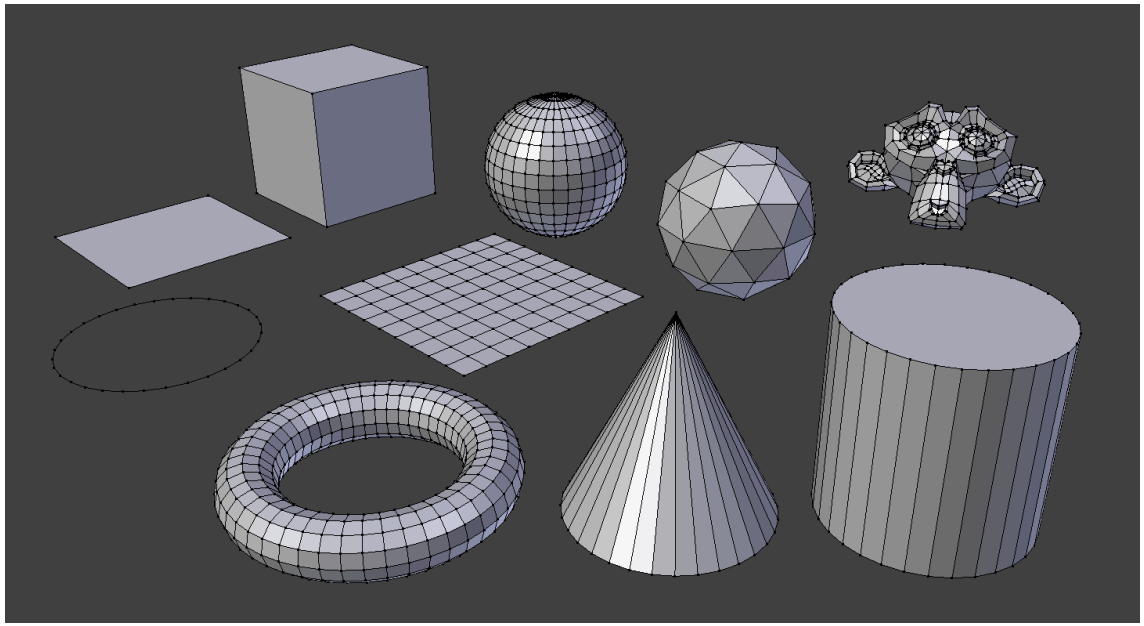
L'élément de base d'un objet 3D est le vertex.

C'est un point dans l'espace associé à des données !



Mesh

Un maillage, ou mesh, est une association de vertices constituant une forme.

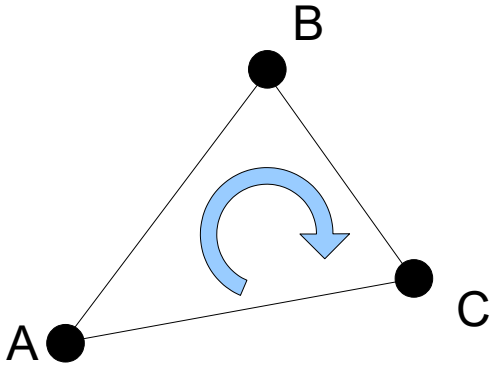


Mesh : Triangle

L'association de vertices la plus répandue est le **triangle**.

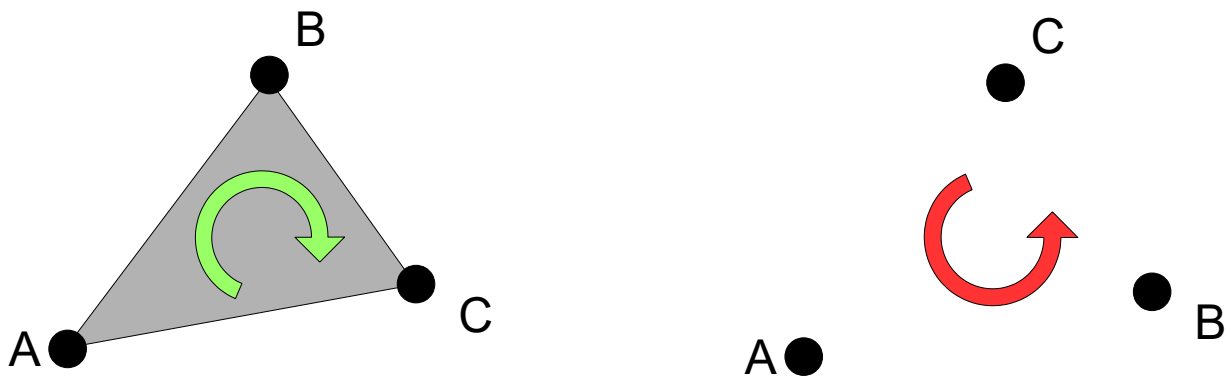
En programmation, le triangle est orienté. Ici :

- ABC nous fait face (clockwise)
- ACB est à l'envers (counter-clockwise).



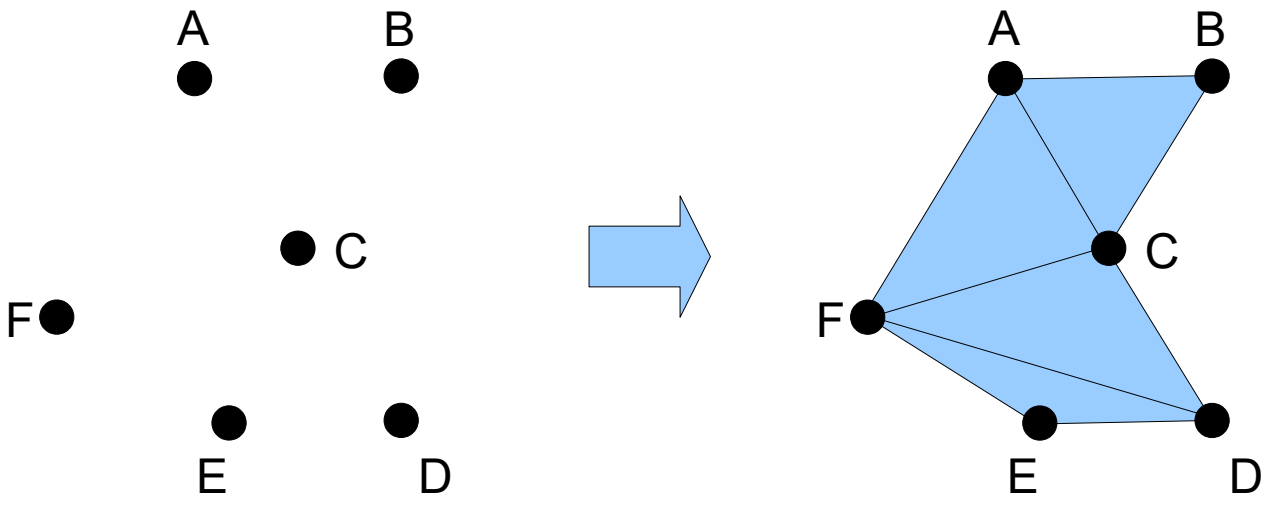
Backface culling

Backface culling : Afin d'optimiser le rendu, les triangles dans la scène ne faisant pas face à la caméra ne sont pas affichés par défaut.



Topologies de mesh

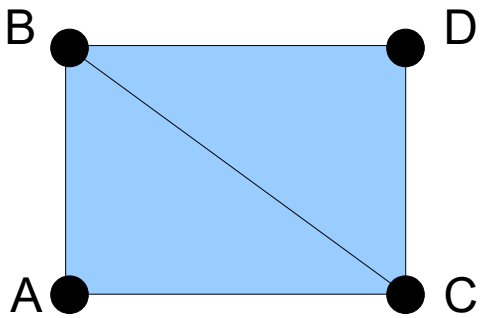
La plupart des meshes se composent uniquement de triangles.



A	B	C	,	A	C	F	,	F	C	D	,	F	D	E

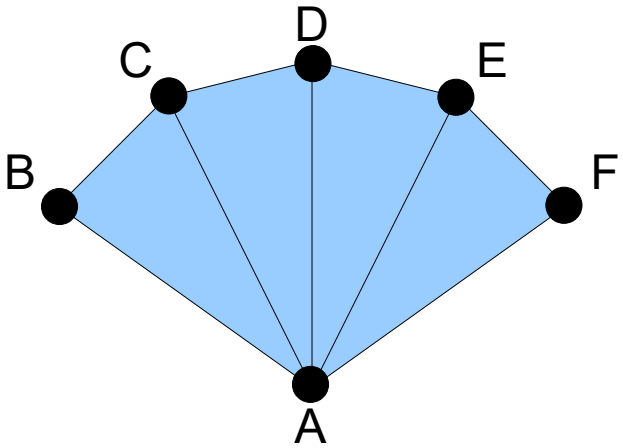
Topologies de mesh

Certaines topologies de mesh permettent de réduire le la quantité d'information utilisée.



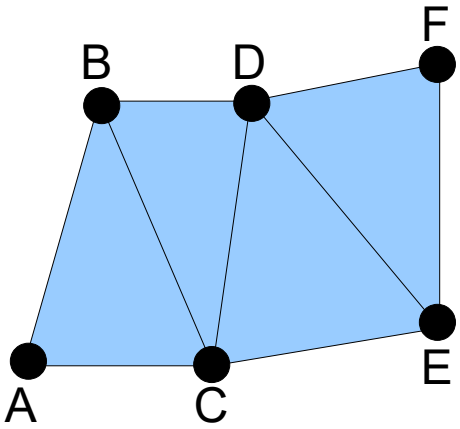
Quad

A	B	C	D



Triangle fan

A	B	C	D	E	F



Triangle strip

A	B	C	D	E	F

Vertex Buffer Object

En programmation, la structure permettant d'assembler des points en meshes est appelée un Vertex Buffer Object, ou **VBO**.

Sous Unity, la classe **Mesh** contient un VBO qui peut être édité via scripts.

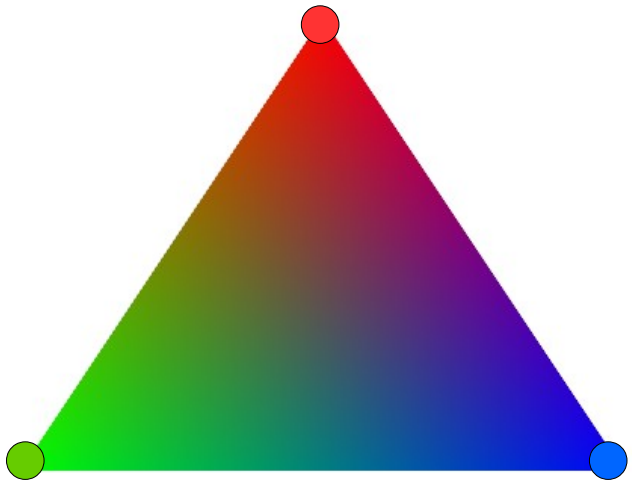
Données de vertex

Un vertex peut porter les informations suivantes utilisées lors du rendu :

- Position (x, y, z),
- Couleur (r, g, b, a),
- Vecteur normal (vers lequel le vertex "fait face"),
- Coordonnées de textures,
- ...

Interpolation

Lors du rendu, les données de points composant un triangle sont interpolées linéairement sur la surface du triangle.



Textures

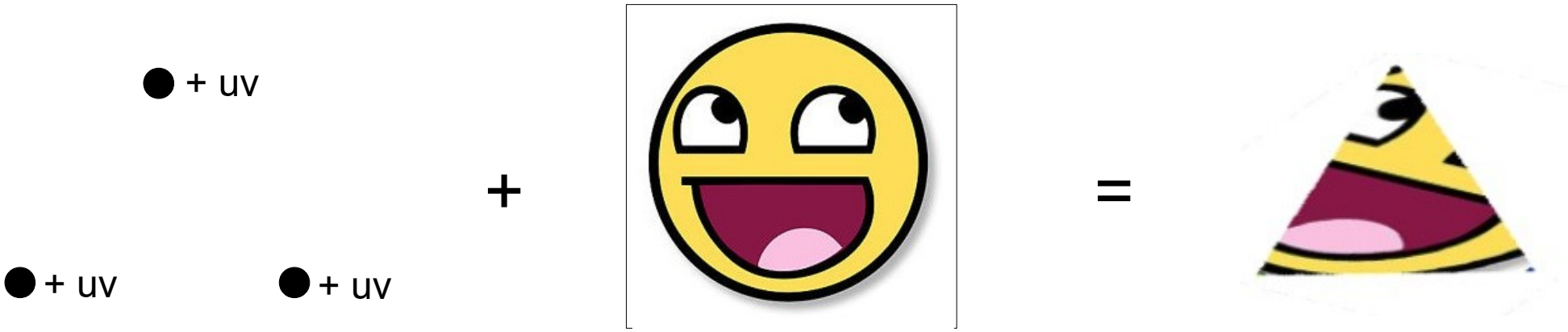
Texture

Une texture est une image utilisée lors du rendu d'un objet 3D.

Elle permet généralement de déterminer une couleur pour chaque pixel qu'occupe un triangle au rendu.

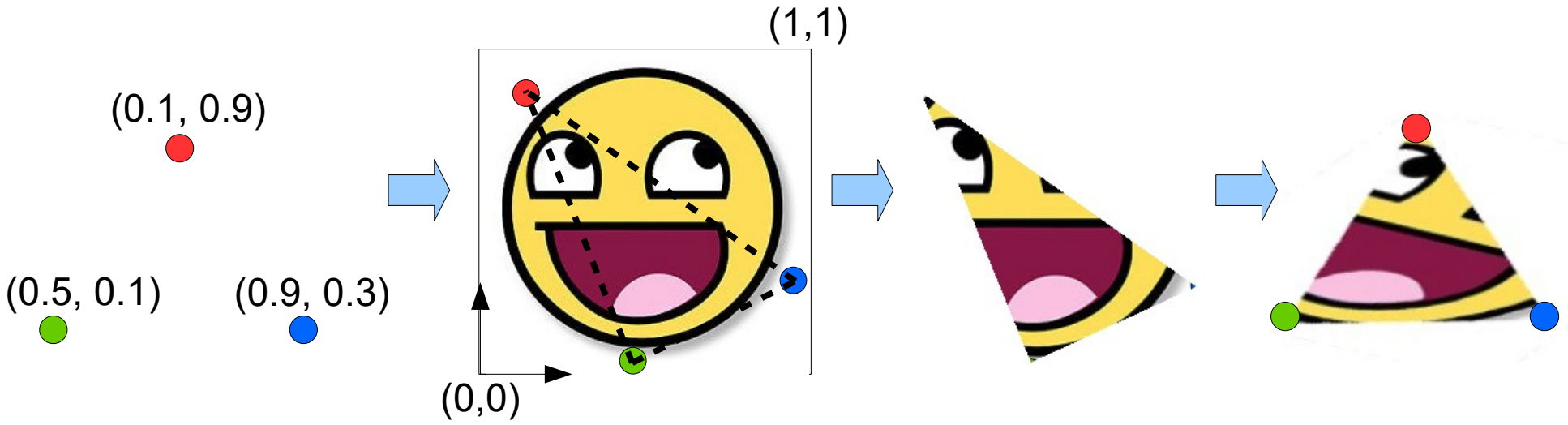
UV mapping

Les coordonnées de textures, dites **UV**, contenues dans les vertices permettent de déterminer quelle partie de la texture utiliser sur le triangle.



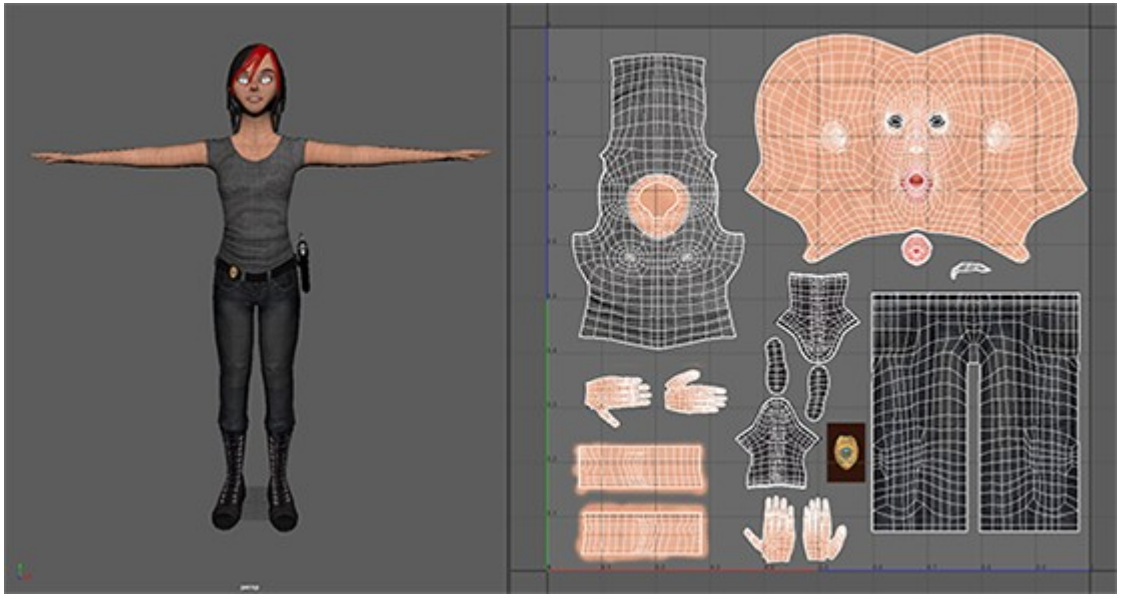
UV mapping

Ces coordonnées de textures UV correspondent à la **position du vertex sur la texture**. Cette position est comprise entre (0,0) et (1,1).



UV mapping

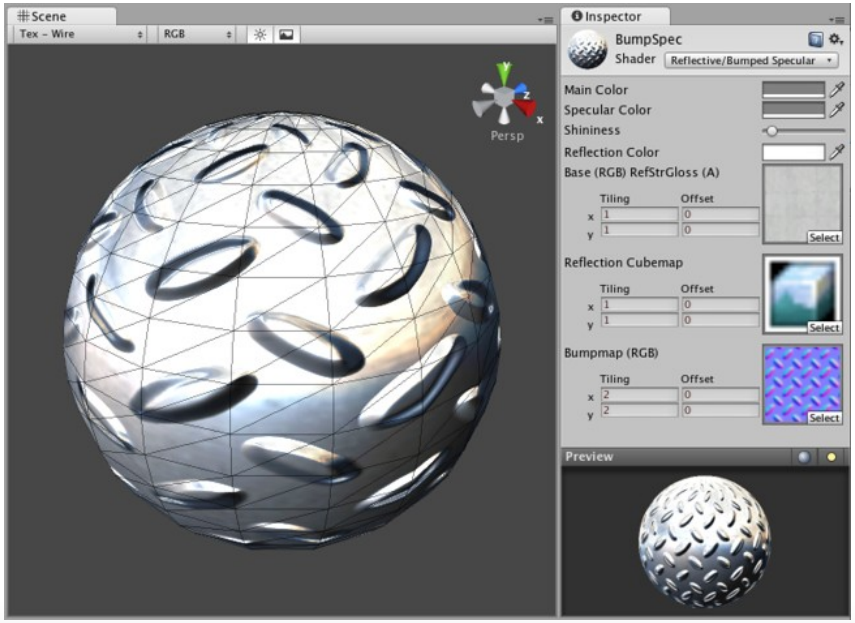
En appliquant ce principe sur chaque triangle d'un mesh, on obtient le résultat suivant :



Materiaux

Material

Un material est une configuration associée à un mesh permettant de contrôler l'aspect de ce dernier lors du rendu.



Material

De manière générale, un material permet de configurer :

- Les textures utilisées pour le rendu
- Une couleur pour l'ensemble de rendu
- Le comportement de l'éclairage sur le mesh

Material : Shader

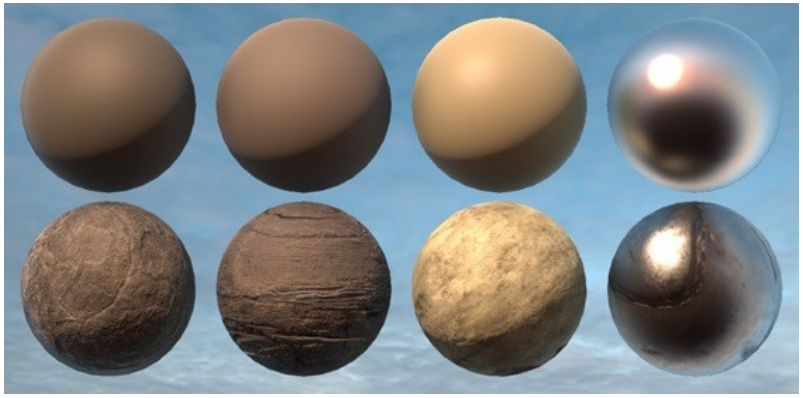
Un material est la configuration d'un shader.

Le shader est un programme exécuté directement par le processeur graphique.

Il permet de traiter un modèle à partir d'une configuration donnée jusqu'à déterminer la couleur de chaque pixel occupé par ce modèle sur la surface de rendu.

Material : PBR

Le type de material actuellement le plus rependu sous Unreal et Unity est le PBR (Physic Based Rendering).

Ce type de material "photoréaliste" cherche à imiter des aspects réels tels que :

- Relief,
- Rugosité,
- Brillance, ...